Telechips 8050 Android 10

Table of Contents:

1. System Check (Hardware Requirements)

To verify system dependencies before building, run the following command:

$ ./system-check.sh

2.) Software Requirements

- Install required packages on Ubuntu 20.04:
    - ➔ sudo apt install -y net-tools make build-essential cmake gcc g++ perl openjdk-11-jdk
    - ➔ sudo apt install -y ctags ncftp chrony diffstat git gnupg flex bison zip curl zlib1g-dev
    - ➔ sudo apt install -y g++-multilib libc6-dev-i386 lib32ncurses-dev libx11-dev lib32z1-dev
    - ➔ sudo apt install -y libgl1-mesa-dev libxml2-utils xsltproc unzip fontconfig sqlite libffi-dev
    - ➔ sudo apt install -y libbz2-dev libreadline-dev libsqlite3-dev libssl-dev lzop dos2unix python2
    - ➔ sudo apt install -y python-dev python3-dev xz-utils tk-dev manpages-dev bzip2 device-tree-compiler
    - ➔ sudo apt install -y gawk wget texinfo chrpath socat cpio python3 python3-pip python3-pexpect
    - ➔ sudo apt install -y debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa
    - ➔ sudo apt install -y libsdl1.2-dev pylint3 xterm python3-subunit mesa-common-dev subversion
    - ➔ sudo apt install -y automake zstd

3.) Test Environment of Telechips

Tested on Ubuntu:

- Ubuntu 20.04.5 LTS
- GNU Make 3.82 or higher
- Python 3.8.10
- Git 2.25.1 or higher
- OpenJDK 9 or higher

4.) BUILD GUIDE (Maincore + Subcore)

(a) Download SDK from **TCC805x_Android10_IVI_3.0.0** in the following Telechips Android Git server.

- $ mkdir mydroid
- $ cd mydroid
- $ **repo init -u ssh://android.telechips.com/android_avn/android/platform/manifest.git - b TCC805x_Android10_IVI_3.0.0**
- $ repo sync

(b) Download and Configure Toolchain for U-Boot:

- Download toolchain from the ARM developer site (version 9.2-2019.12):

  https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-a/downloads/

- on the above website you will find the **Downloads 9.2-2019.12** section**.**

- The information of toolchain to compile U-Boot is **gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz** and **gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz**.

- Extract both files in your home directory.

- The build script already includes the default toolchain path.

(c) Run the Build Script

- You will find build script "**build_script_training_telechip8050.sh"** along with this Document copy and paste the script inside the maincore folder.

- To Build the Maincore as well as the subcore run the below script in terminal inside maincore => **$ ./build_script_training_telechip8050.sh**

- After running the script you will file find the choices , for Maincore Build, press **1** and enter.

  For Maincore make sure these config alredy set as shown below, if not set,set these config selecting option=> "Change Build Config (MainCore)"

  1) | Build Board : tcc8050
  2) | Build Board version : evb_sv1.0
  3) | Android Versrion : Android-10
  4) | Build Platform : arm64
  5) | Option eng/user : userdebug
  6) | Storage : UFS
  7) | Head-Up Display : No
  8) | GPU Virtualization : Yes
  9) | Suspend-To-Ram : No

- **Subcore Build press 2**

  For Subcore make sure these config alredy set as shown below if not set,set these config selecting option=> "Change Build Config (SubCore)"

1) | SDK : tcc805x_android_ivi
2) | MANIFEST : tcc805x_android_subcore.xml
3) | MACHINE : tcc8050-sub
4) | VERSION : release
5) | FEATURES : meta-micom,meta-update,rvc,early-camera
6) | ,kernel-4.14,ufs,gpu-vz,cluster

Note:=>1-From the analysis if we are flashing the binaries and only maincore logo appears an not booting, then it might be due to the problem with subcore images,verify the subcore images if all the images less than 400mb then it might not be build propely then do the below step.

Note:=>2-After subcore build successfull ,check the tools ,buildtools and source-mirror are present or not in subcore.if not present or not fully downloaded.

then copy these files from "Essential_folder_for_Subcore" folder into the subcore,and build again the subcore.

5.) Run Firmware download script

- Run the flashing script from mydroid directory where maincore and subcore are located:
  $ sudo ./FLASH_IMAGE_Telechip_8050_Training.sh
- After flashing, change mode from USB to UFS to confirm that the build and flashing are working properly. Proceed to HIDL demo next.

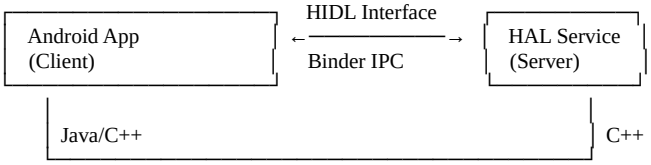
6.) HIDL Echo Demo, Direct App approach (APP -> HAL)

This section demonstrates a basic HIDL Echo HAL communication.

6.1) About HIDL

- HIDL = HAL Interface Definition LanguageIt is a *language and framework* used to define how Android Framework ↔ HAL (hardware layer) talk to each other using Binder IPC.

Key Components:



- **HIDL Communication Models:**

  Binderized (Standard): Client and server in different processes

  Passthrough: Client and server in same process (legacy)

6.2) **Binderised vs Passthrough**

When to Use Each

Mode	When to Use	Reason
Binderized (hwbinder)	For all production/vendor HALs	Safer, process isolation, Android 8+ standard
Passthrough (passthrough)	For debugging / fast testing / early board bring-up	Simpler, no separate process, but unsafe

6.3) **Architecture of our Demo**

[Android App/System]
↓
[HIDL Client] → "I want to call echoString("Hello")"
↓
[Binder IPC] → Carries the request across process boundary
↓
[Echo Service] → Receives request, processes it
↓
[HIDL Implementation] → Creates response "HIDL Echo: Hello"
↓
[Binder IPC] → Carries response back
↓
[Android App/System] → Receives and uses the result

6.4) **Project Structure**

hardware/interfaces/echo/1.0/

├── Android.bp → tells build system to generate HIDL code
├── IEcho.hal → defines the interface (HIDL definition)
├── types.hal → defines custom data types (optional)
└── default/
├── Android.bp → builds HAL service binary
├── Echo.h / Echo.cpp → your implementation of the IEcho interface
├── service.cpp → registers service with hwservicemanager
└── android.hardware.echo@1.0-service.rc → starts service at boot

What Each Part Does

Component	Who Creates It	Purpose
IEcho.hal	You	Defines the HAL interface (echoString() etc.)
hidl-gen output (IEcho.h, BpHwEcho.h, etc.)	Generated during build	Auto-created binder glue code for client ↔ service communication
Echo.h / Echo.cpp	You	Implements the real logic of the HAL
service.cpp	You	Registers the HAL with Android's hwservicemanager
Android.bp	You	Build configuration (to compile everything)
android.hardware.echo@1.0-service.rc	You	Tells Android to start the HAL service on boot
manifest.xml	You	Declares this HAL so framework knows it exists

6.5) **HIDL-GEN tool**

Generate C++ interface

hidl-gen -o hardware/interfaces/echo/1.0/default/ -Lc++-impl -randroid.hardware:hardware/interfaces/ android.hardware.echo@1.0

Generate Android.bp for build

hidl-gen -o hardware/interfaces/echo/1.0/default/ -Landroidbp-impl -randroid.hardware:hardware/interfaces/ android.hardware.echo@1.0

hidl-gen tool ← auto-generates binder proxy + stub classe

=>HIDL Echo Demo Code in Telechips8050 Integration Steps:

- Follow the bellow instruction

  Along with this Doc there is zip/folder name echo copy and paste the file in below location **mydroid/maincore/hardware/interfaces/**

- In this file we have below directory structure

```
echo
└── 1.0
    ├── Android.bp
    ├── default
    │   ├── Android.bp
    │   ├── android.hardware.echo@1.0-service.rc
    │   ├── Echo.cpp
    │   ├── Echo.h
    │   └── service.cpp
    ├── IEcho.hal
    └── types.hal

2 directories, 8 files
```

- **Go to the device.mk file and add**
  Location -> mydroid/maincore/device/telechips/car_tcc8050_arm64/**device.mk**

```
PRODUCT_PACKAGES += android.hardware.echo@1.0-service
```

- **add the service in manifest.xml file**
  mydroid/maincore/device/telechips/car_tcc805x/manifest.xml
  Add your echo HAL block inside the root <manifest> tag, alongside other HALs like audio, graphics, etc.

```
<hal format="hidl">
      <name>android.hardware.echo</name>
      <transport>hwbinder</transport>
      <version>1.0</version>
      <interface>
         <name>IEcho</name>
         <instance>default</instance>
      </interface>
   </hal>
```

- after completing every steps build the maincore using build script and flash the images.

- after flashing check that your service binary(android.hardware.echo@1.0-service) and android.hardware.echo@1.0.so file in below location =>
  1-$ adb shell (get the access of telechips8050)
  1-$ ls /vendor/bin/hw/android.hardware.echo@1.0-service
  2-$ ls /vendor/lib64/android.hardware.echo@1.0.so
  3-and check the manifest file in /vendor/manifest.xml you will see the echo block which we have added.
- Check our echo service is running or not

  console:/ # lshal | grep echo
  1|console:/ # ps -A | grep echo
  1|console:/ # logcat | grep echo

  if service is not running then manually run the service using below command take acces using adb shell or putty.

- console:/ # ./vendor/bin/hw/android.hardware.echo@1.0-service &
- [1] 3321
- console:/ # lshal | grep echo
  DM N android.hardware.echo@1.0::IEcho/default 0/1 3321 183
- console:/ # ps -A | grep echo
  root 3321 1826 37260 4836 binder_ioctl_write_read 0 S android.hardware.echo@1.0-service
- console:/ #
- console:/ # logcat | grep echo
  11-04 14:51:51.035 2063 2063 I android.hardware.echo@1.0-service: === Starting HIDL Echo Service ===
  11-04 14:51:51.035 2063 2063 I ServiceManagement: Registered android.hardware.echo@1.0::IEcho/default(start delay of 19ms)
  11-04 14:51:51.036 2063 2063 I ServiceManagement: Removing namespace from process nameandroid.hardware.echo@1.0-service to echo@1.0-service.
  11-04 14:51:51.036 2063 2063 I android.hardware.echo@1.0-service: === HIDL Echo Service Registered Successfully ===
  11-04 14:51:51.036 2063 2063 I android.hardware.echo@1.0-service: === Service name: android.hardware.echo@1.0::IEcho ===
  From the above log it shows our HAL service is curently in running state.

Now service is started

Note => if service binary android.hardware.echo@1.0-service and android.hardware.echo@1.0.so not found in *vendor locaton then manually flash in* /data/local/tmp/ folder ,you will find these files in out folder in below location
1-maincore/out/target/product/car_tcc8050_arm64/vendor/bin/hw/android.hardware.echo@1.0-service
2- maincore/out/target/product/car_tcc8050_arm64/vendor/lib64/android.hardware.echo@1.0.so
using adb push ,push these files in /data/local/tmp/ and run thes service. Execute the below commnads first.
$ su

$cd /data/local/tmp
$ chmod 644 android.hardware.echo@1.0.so
$ chmod 755 android.hardware.echo@1.0-service

$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/data/local/tmp
$ ./android.hardware.echo@1.0-service &

7.) HIDL Test Application -> HidlTestApp

created an simple Android application for HIDL service Testing in below location
maincore/packages/apps/HidlTestApp/
below is the directory structure ->
along with this doc you will find the **HidlTestApp** folder in below location

maincore/packages/apps/

HidlTestApp
├── Android.bp
├── AndroidManifest.xml
├── res
└── src
└── com
└── telechips
└── hidltest
└── MainActivity.java

5 directories, 3 files

Build this application using below command
Note-> do not build the entire source code build only HidlTestApp application.

```
oem@xrda3:~/Documents/Telchip_8050/mydroid/maincore$ source build/envsetup.sh
oem@xrda3:~/Documents/Telchip_8050/mydroid/maincore$ mmm packages/apps/HidlTestApp/
============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=10
TARGET_PRODUCT=aosp_arm
TARGET_BUILD_VARIANT=eng
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm
TARGET_ARCH_VARIANT=armv7-a-neon
TARGET_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-139-generic-x86_64-Ubuntu-20.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=QTR1.210629.001
OUT_DIR=out
============================================
ninja: no work to do.

#### build completed successfully (1 seconds) ####

oem@xrda3:~/Documents/Telchip_8050/mydroid/maincore$
```

After build complete flashed the apk using adb

$ adb install -r maincore/out/target/product/car_tcc8050_arm64/system/product/app/HidlTestApp/HidlTestApp.apk
or
$ adb install -r maincore/out/target/product/generic/system/product/app/HidlTestApp/HidlTestApp.apk

before launching the application do the below things

```
car_tcc8050_arm64:/ # ./android.hardware.echo@1.0-service &
[1] 5174
car_tcc8050_arm64:/ # getenforce
Enforcing   ->> this will create issue while testing disable selinux policy disable using below commands
                    car_tcc8050_arm64:/ # setenforce 0   -> (Disable SELinux for testing)
```